Laborator 6
Tipuri de date compuse. Inregistrare (RECORD). Colecţie (INDEX-BY TABLE, NESTED TABLE, VARRAY)

I. Obiective

6.1. Înregistrări (RECORD)
6.2. Colecţii (INDEX-BY TABLE, NESTED TABLE, VARRAY)
6.2.1. Tablouri indexate (index-by tables)
6.2.2 Vectori (varray)
6.2.3 Tablouri imbricate

II. Consideraţii teoretice

6.1. Înregistrări (RECORD)

Declararea tipului RECORD se face conform următoarei sintaxe:

```
TYPE nume_tip IS RECORD
(nume_câmp1 {tip_câmp | variabilă%TYPE |
  nume_tabel.coloană%TYPE | nume_tabel%ROWTYPE}
  [ [NOT NULL] {:= | DEFAULT} expresie1],
(nume_câmp2 {tip_câmp | variabilă%TYPE |
  nume_tabel.coloană%TYPE | nume_tabel%ROWTYPE}
  [ [NOT NULL] {:= | DEFAULT} expresie2],…);
```

Oracle9i introduce câteva facilităţi legate de acest tip de date.

- Se poate insera (INSERT) o linie într-un tabel utilizând tipul RECORD.
- Se poate actualiza (UPDATE) o linie într-un tabel utilizând tipul RECORD (cu sintaxa SET ROW)
- se poate regăsi şi returna sau şterge informaţia din clauza RETURNING a comenzilor UPDATE sau DELETE
- dacă în comenzile UPDATE sau DELETE se modifică mai multe linii, atunci pot fi utilizate în sintaxa BULK COLLECT INTO, colecţii de înregistrări.

Probleme rezolvate

1. Să se şteargă angajatul având codul 200 din tabelul EMP. Să se reţină într-o variabilă de tip RECORD codul, numele, salariul şi departamentul acestui angajat (clauza RETURNING). Să se afişeze înregistrarea respectivă. Rollback.

```
DECLARE
  TYPE info_ang_pnu IS RECORD (
    cod_ang NUMBER(4),
    nume VARCHAR2(20),
    salariu NUMBER(8),
    cod_dep NUMBER(4));
  v_info_ang  info_ang_pnu;
BEGIN
       DELETE FROM emp
       WHERE empno =200
       RETURNING empno, ename, sal, deptno INTO
       v_info_ang;
       DBMS_OUTPUT.PUT_LINE('A fost stearsa linia continand valorile ' || v_info_ang.cod_ang ||'
       '||v_info_ang.nume||' ' ||v_info_ang.salariu ||' '
       || v_info_ang.cod_dep) ;
END;
```

2. a) Folosind tipul declarat mai sus, să se adauge o linie în tabelul EMP prin intermediul unei variabile de tip înregistrare iniţializate. Efectuaţi modificările necesare asupra tipului de date, astfel încât această inserare să fie posibilă. La iniţializarea unei variabile de tip record, ţineţi cont de constrângerile NOT NULL definite asupra tabelului EMP.
b) Modificaţi valoarea unei componente a variabilei definite anterior şi actualizaţi conţinutul liniei introduse în tabel.

```
DECLARE
  TYPE info_ang_pnu IS RECORD (
    cod_ang  NUMBER(4):=500,
    nume      VARCHAR2(20):='abc',
    prenume VARCHAR2(20):='john',
    email      emp.email%TYPE:='abc@mail',
    telefon     emp.phone_number%type,
    data        emp.hiredate%TYPE:=SYSDATE,
    job          emp.job_id%TYPE:='SA_REP',
    salariu     NUMBER(8, 2):=1000,
    comision emp.comm%TYPE,
    manager  emp.mgr%TYPE,
    cod_dep NUMBER(4):=30
    );
  v_info_ang  info_ang_pnu;
BEGIN
 --inserare; nu ar fi fost posibila maparea unei variabile de tip RECORD într-o lista
-- explicita de coloane
  INSERT INTO emp
  VALUES v_info_ang;
       DBMS_OUTPUT.PUT_LINE('A fost introdusa linia continand valorile ' ||
          v_info_ang.cod_ang ||' '||v_info_ang.nume||' ' ||v_info_ang.salariu ||' '
      || v_info_ang.cod_dep) ;
--actualizare
  v_info_ang.nume:='smith';
   UPDATE emp
   SET ROW=v_info_ang
   WHERE empno = v_info_ang.cod_ang;
   DBMS_OUTPUT.PUT_LINE('A fost actualizata linia cu valorile ' ||
      v_info_ang.cod_ang ||' '||v_info_ang.nume||' ' ||v_info_ang.salariu ||' '
      || v_info_ang.cod_dep) ;
END;
```

6.2. Colecţii (INDEX-BY TABLE, NESTED TABLE, VARRAY)

Colecţiile permit să fie prelucrate simultan mai multe variabile de acelaşi tip. Fiecare element are un indice unic, care determină poziţia sa în colecţie. În PL/SQL există trei tipuri de colecţii:

- tablouri indexate (index-by tables);
- tablouri imbricate (nested tables);
- vectori (varrays sau varying arrays

Observaţie:

- Tipul index-by table poate fi utilizat numai în declaraţii PL/SQL. Tipurile varray şi nested table pot fi utilizate atât în declaraţii PL/SQL, cât şi în declaraţii la nivelul schemei (de exemplu, pentru definirea tipului unei coloane a unui tabel relaţional).

- Singura diferenţă sintactică între tablourile indexate şi cele imbricate este clauza INDEX BY. Dacă această clauză lipseşte, atunci tipul este tablou imbricat.

Atribute şi metode ale unei colecţii:

Atribut sau metodă	**Descriere**
COUNT	numărul componentelor colecţiei
FIRST	Indicele primului element din tablou
LAST	Indicele ultimului element din tablou
EXISTS	întoarce TRUE dacă există în tablou componenta cu indexul specificat
NEXT	returnează indicele următoarei componente
PRIOR	returnează indicele componentei anterioare
DELETE	şterge una sau mai multe componente.
EXTEND	Adaugă elemente la sfârşit
LIMIT	Numărul maxim de elemente al unei colecţii (pentru vectori), null pentru tablouri imbricate
TRIM	şterge elementele de la sfârşitul unei colecţii

Ultimele 3 metode nu sunt valide pentru index-by tables.

- **bulk bind** permite ca toate liniile unei colecţii să fie transferate simultan printr-o singură operaţie.

• este realizat cu ajutorul comenzii FORALL, ce poate fi folosită cu orice tip de colecţie:

FORALL index IN lim_inf..lim_sup
comanda_sql;

Cursorul SQL are un atribut compus %BULK_ROWCOUNT care numără liniile afectate de iteraţiile comenzii FORALL. %BULK_ROWCOUNT(i) reprezintă numărul de linii procesate de a i-a execuţie a comenzii SQL.

- Regăsirea rezultatului unei interogări în colecţii (înainte de a fi trimisă motorului PL/SQL) se poate obţine cu ajutorul clauzei BULK COLLECT:

…BULK COLLECT INTO nume_colecţie [,nume_colecţie]…

Clauza poate să apară în:

- comenzile SELECT INTO (cursoare implicite),
- comenzile FETCH INTO (cursoare explicite),
- clauza RETURNING INTO a comenzilor INSERT, UPDATE, DELETE

Probleme rezolvate

3. Analizaţi şi comentaţi exemplul următor. Afişaţi valorile variabilelor definite.

```
DECLARE
  TYPE tab_index IS TABLE OF NUMBER
    INDEX BY BINARY_INTEGER;
  TYPE tab_imbri IS TABLE OF NUMBER;
  TYPE vector IS VARRAY(15) OF NUMBER;
  v_tab_index tab_index;
  v_tab_imbri tab_imbri;
  v_vector     vector;
  i INTEGER
; BEGIN
  v_tab_index(1) := 72;
  v_tab_index(2) := 23;
  v_tab_imbri := tab_imbri(5, 3, 2, 8,
  7); v_vector := vector(1, 2);
-- afisati valorile variabilelor definite; exemplu dat pentru
v_tab_imbri i:=v_tab_imbri.FIRST;
WHILE (i <= v_tab_imbri.LAST) LOOP
  DBMS_OUTPUT.PUT_LINE('v_tab_imbri:
  '||v_tab_imbri(i)); i:= v_tab_imbri.NEXT(i);
END
LOOP;
END;
```

6.2.1. Tablouri indexate (index-by tables)

Tabloul indexat PL/SQL are două componente:

- coloană ce cuprinde cheia primară pentru acces la liniile tabloului
- o coloană care include valoarea efectivă a elementelor tabloului.

Declararea tipului TABLE se face respectând următoarea sintaxă:

TYPE nume_tip IS TABLE OF
{tip_coloană | variabilă%TYPE |
nume_tabel.coloană%TYPE [NOT NULL] |
nume_tabel%ROWTYPE}
INDEX BY tip_indexare;

Observaţii:

- Elementele unui tablou indexat nu sunt într-o ordine particulară şi pot fi inserate cu chei arbitrare.
- Deoarece nu există constrângeri de dimensiune, dimensiunea tabloului se modifică dinamic.
- Tabloul indexat PL/SQL nu poate fi iniţializat în declararea sa.
- Un tablou indexat neiniţializat este vid (nu conţine nici valori, nici chei).
- Un element al tabloului este nedefinit atâta timp cât nu are atribuită o valoare efectivă.
- Dacă se face referire la o linie care nu există, atunci se produce excepţia
- NO_DATA_FOUND.
- Pentru inserarea unor valori din tablourile PL/SQL într-o coloană a unui tabel de date se utilizează instrucţiunea INSERT în cadrul unei secvenţe repetitive LOOP.
- Pentru regăsirea unor valori dintr-o coloană a unei baze de date într-un tablou PL/SQL se utilizează instrucţiunea FETCH (cursoare) sau instrucţiunea de atribuire în cadrul unei secvenţe repetitive LOOP.

- Pentru a şterge liniile unui tablou fie se asignează elementelor tabloului valoarea null, fie se declară un alt tablou PL/SQL (de acelaşi tip) care nu este iniţializat şi acest tablou vid se asignează tabloului PL/SQL care trebuie şters. În PL/SQL 2.3 ştergerea liniilor unui tabel se poate face utilizând metoda DELETE.

Probleme rezolvate

4. Să se definească un tablou indexat PL/SQL având elemente de tipul NUMBER. Să se introducă 20 de elemente în acest tablou. Să se afişeze, apoi să se şteargă tabloul utilizând diverse metode.

```
DECLARE
  TYPE tablou_numar IS TABLE OF NUMBER
    INDEX BY PLS_INTEGER;
  v_tablou tablou_numar;
  v_aux      tablou_numar; -- tablou folosit pentru stergere
BEGIN
  FOR i IN 1..20 LOOP
    v_tablou(i) := i*i;
    DBMS_OUTPUT.PUT_LINE(v_tablou(i));
  END LOOP;
  --v_tablou := NULL;
  --aceasta atribuire da eroarea PLS-00382
  FOR i IN v_tablou.FIRST..v_tablou.LAST LOOP -- metoda 1 de
    stergere v_tablou(i) := NULL;
  END LOOP;
--sau
  v_tablou := v_aux;   -- metoda 2 de stergere
--sau
  v_tablou.delete; --metoda 3 de stergere
  DBMS_OUTPUT.PUT_LINE('tabloul are ' || v_tablou.COUNT
                    || ' elemente');
END;
```

5. Să se definească un tablou de înregistrări având tipul celor din tabelul dept. Să se iniţializeze un element al tabloului şi să se introducă în tabelul dept. Să se şteargă elementele tabloului.

```
DECLARE
  TYPE dept_table_type IS TABLE OF dept%ROWTYPE
            INDEX BY BINARY_INTEGER;
  dept_table  dept_table_type;
  i               NUMBER;
BEGIN
```

```
 IF dept_table.COUNT <>0 THEN
   i := dept_table.LAST+1;
   ELSE i:=1;
 END IF;
 dept_table(i).deptno := 92;
 dept_table(i).department_name := 'NewDep';
 dept_table(i).location_id := 2700;
 INSERT INTO dept(deptno, dname, loc)
 VALUES (dept_table(i).deptno, dept_table(i).dname, dept_table(i).loc);
 -- sau folosind noua facilitate Oracle9i
 -- INSERT INTO dept
 -- VALUES  dept_table(i);
 dept_table.DELETE; -- sterge toate elementele
 DBMS_OUTPUT.PUT_LINE('Dupa aplicarea metodei
    DELETE sunt '||TO_CHAR(dept_table.COUNT)||'
    elemente');
END;
```

6.2.2 Vectori (varray)

- Vectorii (varray) sunt structuri asemănătoare vectorilor din limbajele C sau Java.
- Vectorii au o dimensiune maximă (constantă) stabilită la declarare. În special, se utilizează pentru modelarea relaţiilor one-to-many, atunci când numărul maxim de elemente din partea „many“ este cunoscut şi ordinea elementelor este importantă.
- Fiecare element are un index, a cărui limită inferioară este 1.

Tipul de date vector este declarat utilizând sintaxa:

TYPE nume_tip IS
{VARRAY | VARYING ARRAY} (lungime_maximă)
OF tip_elemente [NOT NULL];.

Probleme rezolvate

6. Analizaţi şi comentaţi exemplul următor.

```
DECLARE
   TYPE secventa IS VARRAY(5) OF
    VARCHAR2(10); v_sec secventa := secventa ('alb',
    'negru', 'rosu',
                        'verde');
BEGIN
 v_sec (3) := 'rosu';
 v_sec.EXTEND; -- adauga un element
 null v_sec(5) := 'albastru';
 -- extinderea la 6 elemente va genera eroarea ORA-
 06532 v_sec.EXTEND;
END;
```

7. Să se scrie un bloc care măreşte salariile angajaţilor din departamentul 50 cu 10%, în cazul în care salariul este mai mic decât 5000. Se va utiliza un vector corespunzător codurilor angajaţilor. Se cer 3 soluţii.

Soluţia 1:

```
DECLARE
      TYPE t_id IS VARRAY(100) OF emp.empno%TYPE
      ; v_id t_id := t_id();
BEGIN
FOR contor IN (SELECT * FROM emp) LOOP
IF contor.deptno =50 AND contor.sal < 5000
THEN V_id.extend;
V_id(v_id.COUNT) := contor.empno;
END IF;
END LOOP;
      FOR contor IN 1..v_id.COUNT LOOP
                UPDATE emp
                SET sal = sal *1.1
                WHERE empno = v_id (contor);
      END LOOP;
END;
```

Soluţia 2 (varianta FORALL):

```
DECLARE
      TYPE t_id IS VARRAY(100) OF emp.empno%TYPE
      ; v_id t_id := t_id();
BEGIN
FOR contor IN (SELECT * FROM emp) LOOP
IF contor. Deptno =50 AND contor.sal < 5000
THEN V_id.extend;
V_id(v_id.COUNT) := contor.empno;
END IF;
END LOOP;
      FORALL contor IN 1..v_id.COUNT
                UPDATE emp
                SET sal = sal *1.1
                WHERE empno = v_id (contor);
END;
```

Observație: Prin comanda FORALL sunt trimise toate datele pe server, executându-se apoi o singură comandă SELECT, UPDATE etc.

Soluţia 3 (varianta BULK COLLECT):

```
DECLARE
      TYPE t_id IS VARRAY(100) OF emp.empno%TYPE
      ; v_id t_id := t_id();
BEGIN
SELECT empno
BULK COLLECT INTO v_id
FROM emp
WHERE  deptno =50 AND sal < 5000;
      FORALL contor IN 1..v_id.COUNT
                UPDATE emp
                SET sal = sal *1.1
                WHERE empno = v_id (contor);
END;
```

6.2.3 Tablouri imbricate

Tablourile imbricate (nested table) sunt tablouri indexate a căror dimensiune nu este stabilită.

- folosesc drept indici numere consecutive ;
- sunt asemenea unor tabele cu o singură coloană;
- nu au dimensiune limitată, ele cresc dinamic;
- iniţial, un tablou imbricat este dens (are elementele pe poziţii consecutive) dar pot apărea spaţii goale prin ştergere ;
- metoda NEXT ne permite să ajungem la următorul element ;
- pentru a insera un element nou, tabloul trebuie extins cu metoda EXTEND(nr_comp) ;

Un tablou imbricat este o mulţime neordonată de elemente de acelaşi tip. Valorile de acest tip:

- pot fi stocate în baza de date,
- pot fi prelucrate direct în instrucţiuni SQL
- o au excepţii predefinite proprii.

Comanda de declarare a tipului de date tablou imbricat are sintaxa:

```
TYPE nume_tip IS TABLE OF tip_ elemente [NOT NULL];
```

Pentru adaugarea de linii intr-un tablou imbricat, acesta trebuie sa fie initializat cu ajutorul constructorului.

- PL/SQL apelează un constructor numai în mod explicit.
- Tabelele indexate nu au constructori.
- Constructorul primeşte ca argumente o listă de valori numerotate în ordine, de la 1 la numărul de valori date ca parametrii constructorului.
- Dimensiunea iniţială a colecţiei este egală cu numărul de argumente date în constructor, când aceasta este iniţializată.
- Pentru vectori nu poate fi depăşită dimensiunea maximă precizată la declarare.
- Atunci când constructorul este fără argumente, va crea o colectie fără nici un element (vida), dar care are valoarea not null.

Probleme rezolvate

8. Să se declare un tip tablou imbricat şi o variabilă de acest tip. Iniţializaţi variabila şi afişaţi conţinutul tabloului, de la primul la ultimul element şi invers.

```
DECLARE
  TYPE CharTab IS TABLE OF CHAR(1);
  v_Characters CharTab :=
    CharTab('M', 'a', 'd', 'a', 'm', ',', ' ',
            'I', "", 'm', ' ', 'A', 'd', 'a', 'm');
  v_Index INTEGER;
BEGIN
  v_Index := v_Characters.FIRST;
  WHILE v_Index <= v_Characters.LAST LOOP
    DBMS_OUTPUT.PUT(v_Characters(v_Index))
    ; v_Index := v_Characters.NEXT(v_Index);
  END LOOP;
  DBMS_OUTPUT.NEW_LINE;
  v_Index := v_Characters.LAST;
  WHILE v_Index >= v_Characters.FIRST LOOP
    DBMS_OUTPUT.PUT(v_Characters(v_Index))
    ; v_Index := v_Characters.PRIOR(v_Index);
  END LOOP;
  DBMS_OUTPUT.NEW_LINE;
END;
```

9. Creaţi un tip tablou imbricat, numit NumTab. Afişaţi conţinutul acestuia, utilizând metoda EXISTS. Atribuiţi valorile tabloului unui tablou index-by. Afişaţi şi acest tablou, in ordine inversă.

```
DECLARE  -- cod partial, nu sunt declarate tipurile!
  v_NestedTable NumTab := NumTab(-7, 14.3, 3.14159, NULL, 0);
  v_Count BINARY_INTEGER := 1;
  v_IndexByTable IndexByNumTab;
BEGIN
    LOOP
    IF v_NestedTable.EXISTS(v_Count) THEN
      DBMS_OUTPUT.PUT_LINE(
        'v_NestedTable(' || v_Count || '): ' ||
        v_NestedTable(v_Count));
      v_IndexByTable(v_count) :=
      v_NestedTable(v_count); v_Count := v_Count + 1;
    ELSE
      EXIT;
    END IF;
  END LOOP;
-- atribuire invalida
--  v_IndexByTable := v_NestedTable;
  v_Count := v_IndexByTable.COUNT;
  LOOP
    IF v_IndexByTable.EXISTS(v_Count)
      THEN DBMS_OUTPUT.PUT_LINE(
        'v_IndexByTable(' || v_Count || '): '
        || v_IndexByTable(v_Count));
      v_Count := v_Count - 1;
ELSE
      EXIT;
    END IF;
  END LOOP;
END;
END;
```

III. Desfășurarea lucrării

Testare secvențelor în PL/SQL pe baza structurilor de baze de date create personal, utilizând exemplele prezentate.